

Agile Data Science

What it looks like in a real job

Raúl Cumplido

Who am I

Raúl Cumplido

- Scientific Software Engineer at **QuantStack**
- 25 years of applied experience on engineering worlds
- **Apache Software Foundation** member, **Apache Arrow** PMC member and release manager
- Work mostly on **PyArrow**, Arrow C++, CI/CD and build systems
- Python developer who ended up maintaining a lot of infrastructure
- Associate Professor here at UB

What I actually do

Most of my day is not writing data science code. It is:

- Making sure code can be **built** and **shipped** on many platforms
- Making sure changes don't **break** anything (CI/CD)
- Making sure someone else can **reproduce** what I did
- Reviewing other people's changes

This is the part of the job nobody teaches you, and the part you will spend most of your time on.

Who are you?

What this part of the course is about

You will already know learn to train a model in a notebook on your laptop.

The problem is everything around it:

- Where does the **code** live? Who else can change it?
- Where does the **data** live?
- Does it run on **someone else's machine**?
- Can you tell me which **version** of the model produced that number?
- What happens when it needs to run **every day**, not once?

My sessions

Date	Session
21/09/2026	GitHub: Tasks, Projects, Branches, VS Code
28/09/2026	Docker
05/10/2026	Storage
19/10/2026	Cloud Infrastructure
26/10/2026	MLOps and mlflow
28/10/2026	DVC and Hugging Face (1h)

Then the project sprints, shared with Eloi.

How the sessions fit together

```
Git/GitHub → collaborate on code
  ↓
Docker      → same environment everywhere
  ↓
Storage     → where the data lives, in which format
  ↓
Cloud       → run it somewhere that is not your laptop
  ↓
MLOps       → track experiments, version data and models
```

Each session builds on the previous one. You will use all of them in the project.

What you need

- A laptop you can install things on
- A **GitHub account** (we set it up today)
- **Git** installed locally
- **Visual Studio Code** (or your editor of choice)
- Docker, from session 2 onwards

Today

First hour: how Git and GitHub work, and why. Plus where you actually write the code: editors, IDEs and VS Code.

Second hour: you install and set everything up:

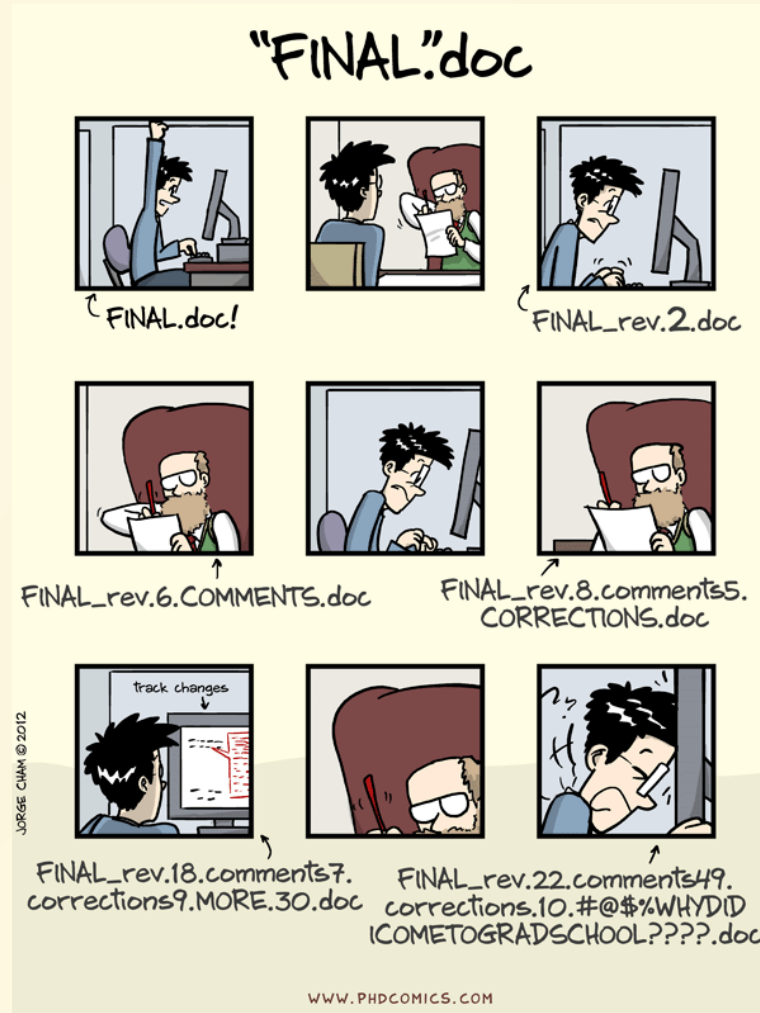
- Git
- GitHub
- VS Code

And use it. By the end of the class you will have pushed code to GitHub and opened a pull request.

Part 1

Git and GitHub

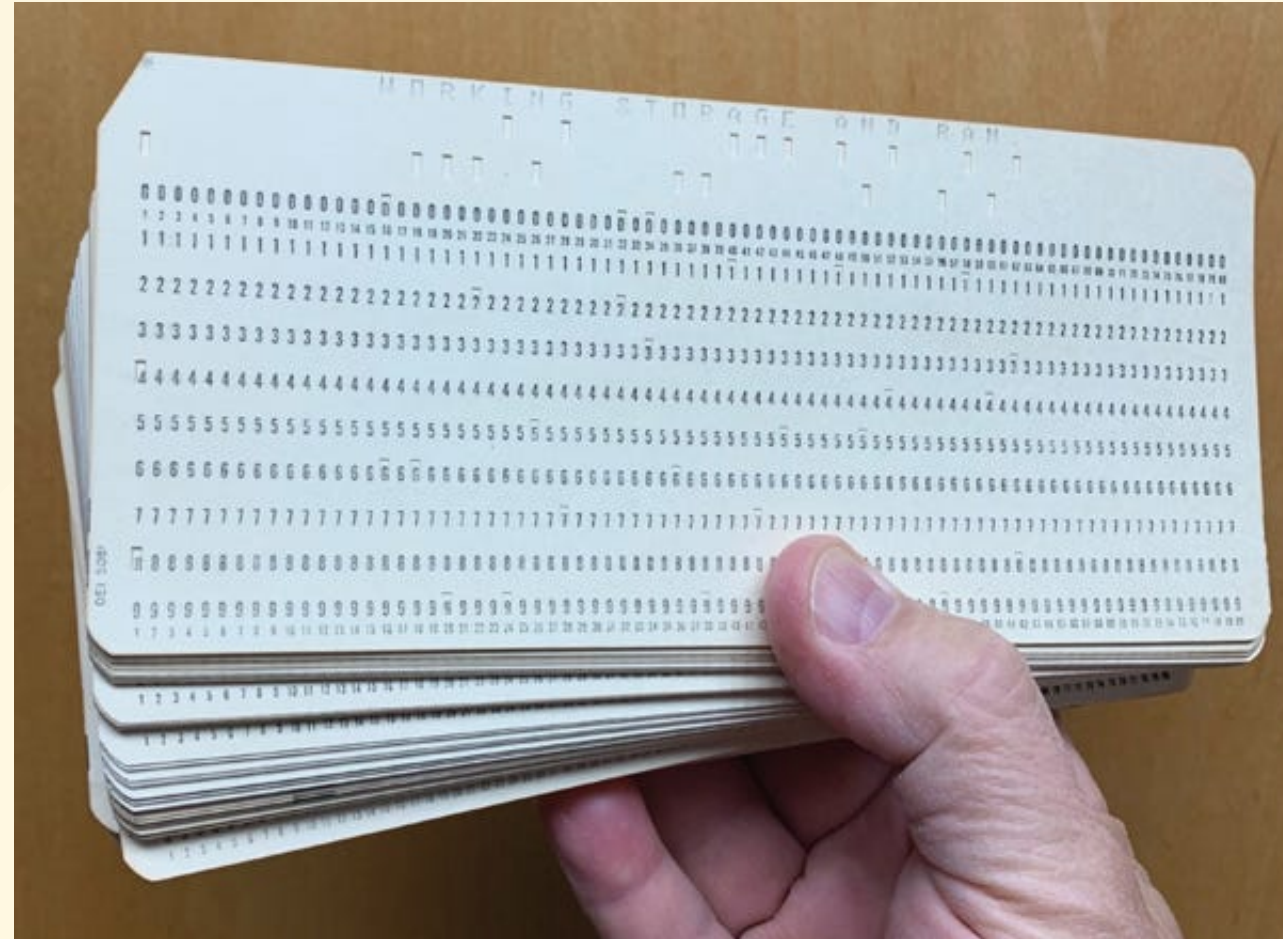
Version Control



"Piled Higher and Deeper" by Jorge Cham, <http://www.phdcomics.com>

Version Control history

- Punchcards (not two people working on the same code)
- FTP/Disks shared files



Version Control System

Also known as **Revision control** or **source control (scm)** is the management of changes to documents, programs, and other information stored as computer files.

They are used in virtually all software development and in all environments, by everyone and everywhere VCS can be used on almost any digital content, so it is not only restricted to software development, and is also very useful for manuscript files, figures, data and notebooks!

CVCS

- Centralised Version Control Systems:
 - Uses a single central server to store the repository and all file versions.
 - Developers work with a snapshot copy of the filesystems on a given version.
 - Examples: **Subversion** / **CVS**

DVCS

- Distributed Version Control Systems
 - Uses multiple copies and each copy could act as a "server" give the correct configuration (set up ssh for example)
 - Each developer has a full copy of the repository, allowing for offline work and greater flexibility.
 - Examples: **Mercurial** / **Git**

Git

Git is a free and open-source distributed version control system designed to handle everything from small to very large projects with speed and efficiency.

Git

Key characteristics:

- Created by Linus Torvalds in 2005
- Distributed: Every developer has a full copy of the project history
- Fast and lightweight
- Tracks changes in files over time
- Enables collaboration between multiple developers

Git repository

A **Git repository** (or "repo") is a directory that contains your project files and the entire history of changes made to those files.

Components of a Git repository:

- **Working directory:** Your current files
- **.git folder:** Contains all version history and Git metadata
- **Staging area:** Temporary area where changes are prepared for commit

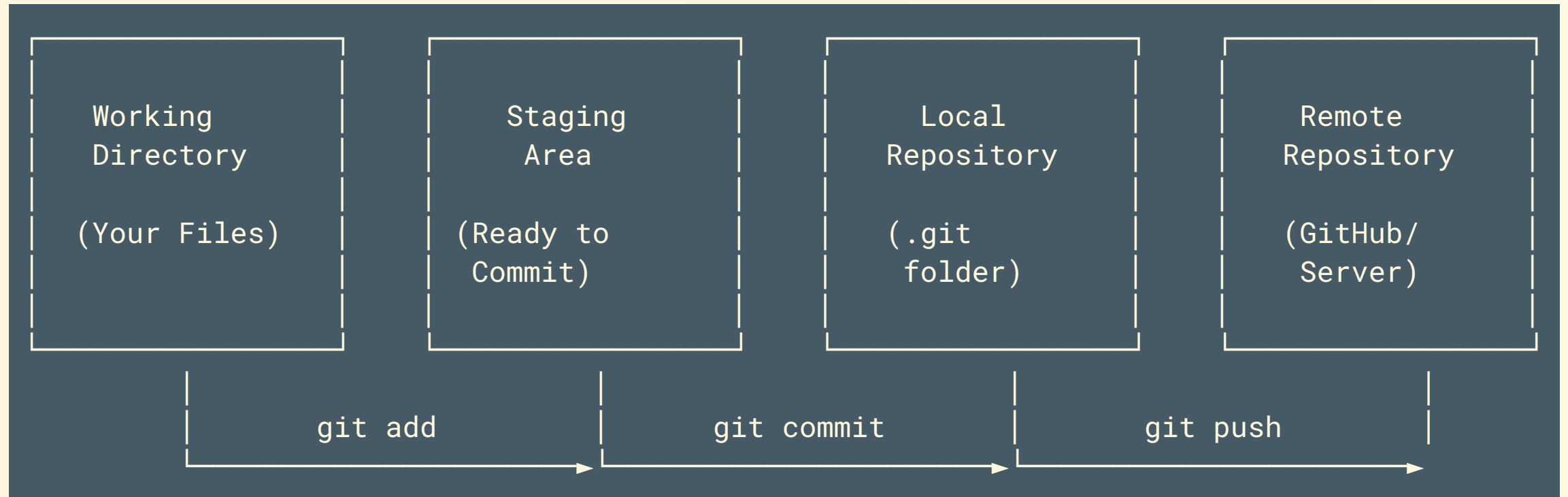
Git repository

Types:

- **Local repository:** On your computer
- **Remote repository:** On a server (like GitHub, GitLab)

Git's Four Areas

Understanding where your files live in the Git workflow.



Git's Four Areas

1. Working Directory

- Your actual files on disk that you can see and edit
- Changes here are "untracked" until staged

2. Staging Area

- Files prepared for the next commit (after `git add`)
- Think of it as a "preview" of your next commit

Git's Four Areas

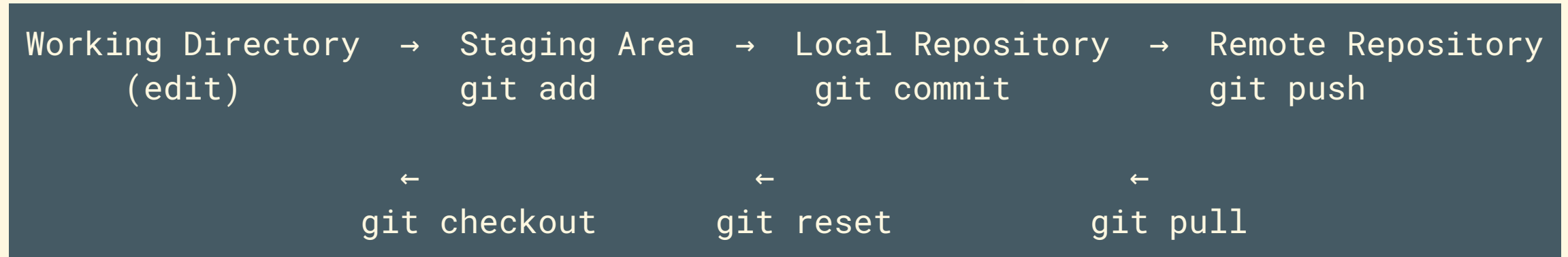
3. Local Repository

- Committed changes stored in your `.git` folder
- Your complete project history on your computer

4. Remote Repository

- The version stored on GitHub/server
- Shared with other developers

Git Workflow Between Areas



Key commands:

- `git add` - Working Directory → Staging Area
- `git commit` - Staging Area → Local Repository
- `git push` - Local Repository → Remote Repository
- `git pull` - Remote Repository → Local Repository + Working

Git commit

A **commit** is a record of specific changes made to your project files.

What a commit contains:

- Changes made to files since the last commit
- A unique identifier (SHA hash)
- Author information and timestamp
- A commit message describing the changes

Git commit

Basic workflow:

1. Make changes to files
2. Stage changes: `git add <file>`
3. Commit changes: `git commit -m "message"`

Git commit

Best practices:

- Write clear, descriptive commit messages
- Make atomic commits (one logical change per commit)

Git branch

A **branch** is a lightweight, movable pointer to a specific commit that allows you to work on different features or experiments in parallel.

Key concepts:

- **main/master**: The default branch (primary line of development)
- **HEAD**: Points to the current branch you're working on
- Branches are cheap and fast to create

Git branch

Common commands:

- `git branch` - List all branches
- `git branch <name>` - Create a new branch
- `git checkout <name>` - Switch to a branch
- `git checkout -b <name>` - Create and switch to new branch
- `git switch <name>` - Modern way to switch branches

Git branch

Why use branches?

- Isolate feature development
- Experiment without affecting main code
- Enable parallel development

Git merge

Merging is the process of combining changes from one branch into another branch.

Basic workflow:

1. Switch to target branch: `git checkout main`
2. Merge feature branch: `git merge feature-branch`

There are different kinds of merge, and a whole alternative (`rebase`).
We look at those at the end, as extra material.

Git conflict - What is it?

A **conflict** occurs when Git cannot automatically merge changes from different branches.

When conflicts happen:

- Two branches modify the same lines in a file
- One branch deletes a file while another modifies it
- Different changes to the same file in a 3-way merge

Git conflict - What it looks like

```
def calculate_total(items):  
<<<<<< HEAD  
    # Calculate total with tax included  
    total = sum(item.price for item in items)  
    return total * 1.08 # 8% tax  
=====  
    # Calculate total price with discount  
    subtotal = sum(item.price for item in items)  
    discount = subtotal * 0.1 # 10% discount  
    return subtotal - discount  
>>>>>> feature-branch
```

Git conflict - Resolution

Steps to resolve:

1. Open the conflicted file in your editor
2. Choose which changes to keep (or combine both)
3. Remove the conflict markers
4. Test your code
5. Add the resolved file: `git add filename`
6. Complete the merge: `git commit`

Git commands you will use today

- `git status` - Show working directory status
- `git diff` - Show differences between commits/files
- `git log` - Show commit history
- `git add` - Stage changes for commit
- `git commit` - Create a new commit
- `git push` - Upload changes to remote repository
- `git pull` - Download and merge changes from remote
- `git clone` - Copy a remote repo to your computer
- `git switch -c <name>` - Create and switch to a new branch

Git commands

```
raulcd@rhoegar:arrow [(GH-43416)]$ git status
```

```
On branch GH-43416
```

```
Your branch is up to date with 'origin/GH-43416'.
```

```
Changes not staged for commit:
```

```
(use "git add <file>..." to update what will be committed)
```

```
(use "git restore <file>..." to discard changes in working directory)
```

```
    modified:   ci/docker/python-wheel-musllinux-test.dockerfile
```

```
    modified:   ci/vcpkg/ports.patch
```

```
Untracked files:
```

```
(use "git add <file>..." to include in what will be committed)
```

```
    test_orc_exceptions.cc
```

```
no changes added to commit (use "git add" and/or "git commit -a")
```

Git commands

```
raulcd@rhoegar:arrow [(GH-43416)]$ git diff ci/docker/python-wheel-musllinux-test.dockerfile
diff --git a/ci/docker/python-wheel-musllinux-test.dockerfile b/ci/docker/python-wheel-musllinux-test.dockerfile
index 601ef886bc..29a55f24d5 100644
--- a/ci/docker/python-wheel-musllinux-test.dockerfile
+++ b/ci/docker/python-wheel-musllinux-test.dockerfile
@@ -29,11 +29,6 @@ RUN apk add --no-cache \
     ENV TZDIR=/usr/share/zoneinfo
     RUN cp /usr/share/zoneinfo/Etc/UTC /etc/localtime

-# pandas doesn't provide wheel for aarch64 yet, so cache the compiled
-# test dependencies in a docker image
-COPY python/requirements-wheel-test.txt /arrow/python/
-RUN pip install -r /arrow/python/requirements-wheel-test.txt
-
# Install the GCS testbench with the system Python
COPY ci/scripts/install_gcs_testbench.sh /arrow/ci/scripts/
ENV PIPX_PYTHON=/usr/bin/python3 PIPX_PIP_ARGS=--prefer-binary
```

Git commands

```
raulcd@rhoegar:arrow [(GH-43416)]$ git log -1  
commit 4a2e8eef98b9ca929bad9b1a45ef78a27aa58a42 (HEAD -> GH-43416-4, origin/GH-43416)  
Author: Raúl Cumplido <raulcumplido@gmail.com>  
Date: Sat Sep 27 13:28:12 2025 +0200
```

Set CMAKE Generator to Visual Studio 17 for Windows wheels

Git commands

```
raulcd@rhoegar:arrow [(GH-43416)]$ git add ci/docker/python-wheel-musllinux-test.dockerfile
```

```
raulcd@rhoegar:arrow [(GH-43416)]$ git status
```

On branch GH-43416

Your branch is up to date with 'origin/GH-43416'.

Changes to be committed:

(use "git restore --staged <file>..." to unstage)

modified: ci/docker/python-wheel-musllinux-test.dockerfile

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)

(use "git restore <file>..." to discard changes in working directory)

modified: ci/vcpkg/ports.patch

Untracked files:

(use "git add <file>..." to include in what will be committed)

test_orc_exceptions.cc

no changes added to commit (use "git add" and/or "git commit -a")

Git commands

```
raulcd@rhoegar:arrow [(GH-43416)]$ git commit -m "Remove pandas requirements from musllinux test dockerfile"
[GH-43416 5a3b2f1] Remove pandas requirements from musllinux test dockerfile
1 file changed, 5 deletions(-)

raulcd@rhoegar:arrow [(GH-43416)]$ git status
On branch GH-43416
Your branch is ahead of 'origin/GH-43416' by 1 commit.
(use "git push" to publish your local commits)

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   ci/vcpkg/ports.patch

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        test_orc_exceptions.cc

no changes added to commit (use "git add" and/or "git commit -a")
```

GitHub

GitHub is a cloud-based hosting service for Git repositories that adds collaboration features.



GitHub

Key features:

- Remote repository hosting
- Web-based Git repository management
- Collaboration tools (pull requests, issues, discussions)
- Project management features
- Continuous Integration/Deployment (CI/CD)
- Documentation hosting (GitHub Pages)

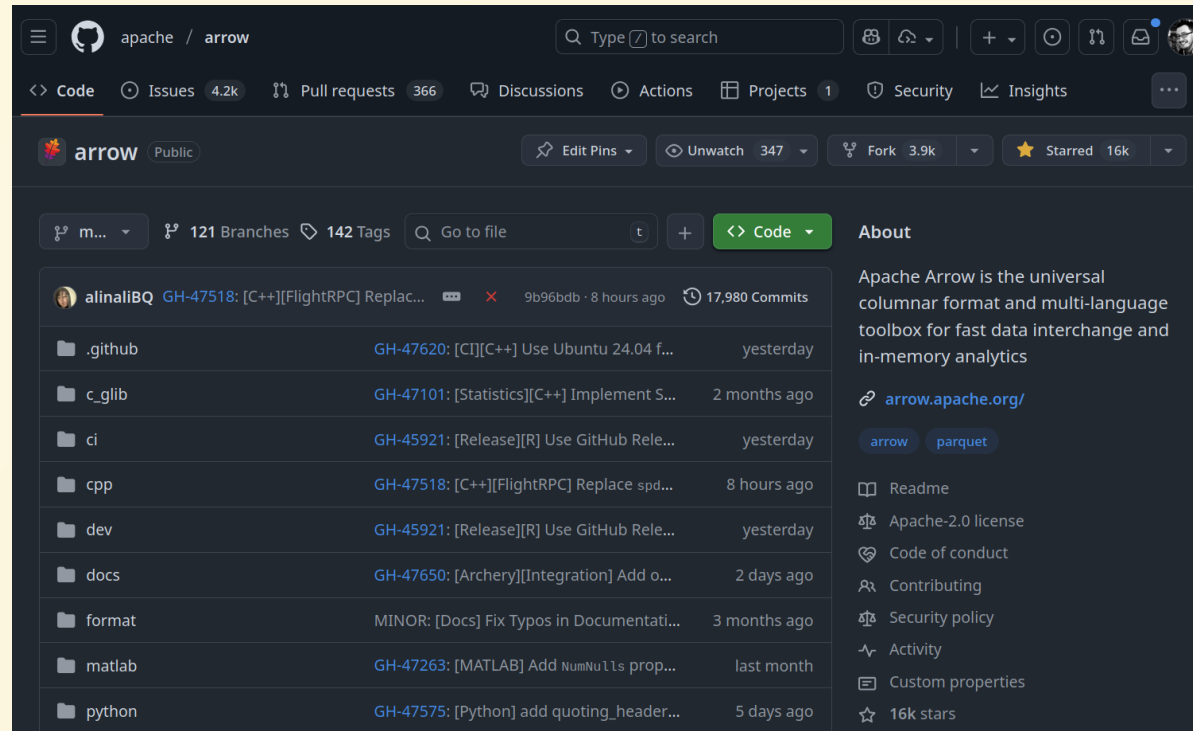
GitHub

Why use GitHub?

- Backup your code in the cloud
- Collaborate with team members
- Contribute to open source projects
- Portfolio for your work

GitHub Repositories

A Git repository hosted on GitHub that multiple people can access.



GitHub Remote Repositories

Key concepts:

- **Origin:** Default name for your main remote repository
- **Push:** Upload your local changes to the remote
- **Pull:** Download changes from the remote to your local repo (equivalent to fetch + merge)
- **Fetch:** Download changes but don't merge them yet

GitHub Remote Repositories

Common commands:

- `git remote -v` - Show remote repositories
- `git push origin main` - Push to main branch
- `git pull origin main` - Pull from main branch
- `git clone <url>` - Copy a remote repo to your computer

GitHub Fork

Fork: Create your own copy of someone else's repository on GitHub.
Can be an organization.

Enables collaboration for a workflow to work on your own copy of the repository via Pull Requests.

Example Open Source contributions.

- Click "Fork" button on GitHub
- You own the forked copy
- Fork → modify → pull request

GitHub Pull Request

A **Pull Request (PR)** is a request to merge your changes into another branch or repository.

What it does:

- Proposes changes to the main codebase
- Enables code review and discussion
- Shows the differences (diff) between branches
- Allows testing before merging

GitHub Pull Request

Typical workflow:

1. Create a feature branch
2. Make changes and commit them
3. Push branch to GitHub
4. Create a pull request
5. Team reviews the code
6. Make requested changes (if needed)
7. Pull request gets merged

GitHub Pull Request

The screenshot shows a GitHub Pull Request (PR) for the 'apache / arrow' repository. The PR title is 'GH-42137 [CI][Python] Add Python Windows GitHub Action and remove AppVeyor #47567'. It is marked as 'Merged' and shows that 'raulcd' merged 35 commits into 'apache:main' from 'raulcd:try-windows-gha' last week. The PR includes 25 conversations, 35 commits, 27 checks, and 6 files changed. A summary of changes shows +237 additions and -328 deletions.

raulcd commented 2 weeks ago • edited by kou

Rationale for this change

There is currently a single CI job on AppVeyor. This is the only job on Windows for Python and it usually fails due to CI issues rather than Python real issues. We have more experience with GitHub actions and debugging AppVeyor is usually challenging.

What changes are included in this PR?

Create a Python Windows job that runs on GitHub actions. The job builds Arrow CPP and PyArrow and it tests PyArrow.

Also removes the CI related files for AppVeyor

Are these changes tested?

Yes, via CI.

Are there any user-facing changes?

No

• GitHub Issue: [\[CI\]\[Python\] Migrate AppVeyor job to GitHub actions #42137](#)

Reviewers

- pitrou** ✓
- kou** ✓
- assignUser** ⚠
- Jonkeane** ⚠

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

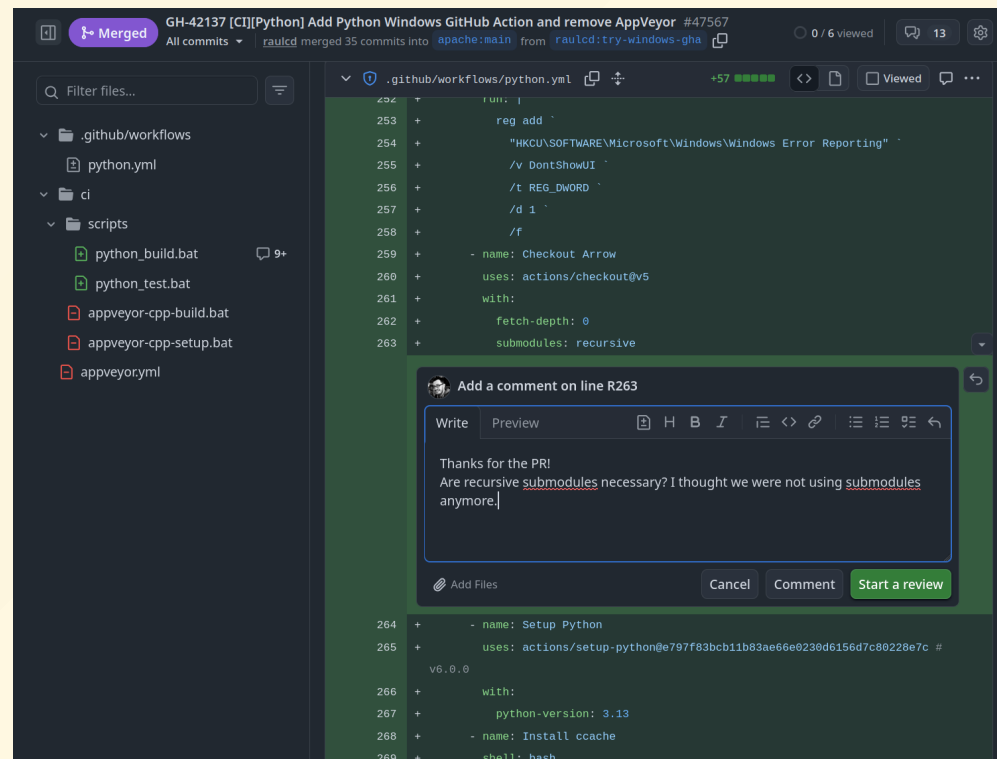
None yet

Milestone

No milestone

GitHub Review

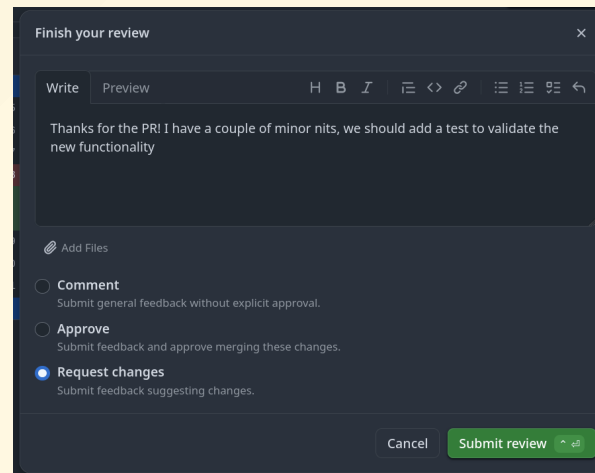
Code Review is the process of examining code changes before they're merged.



GitHub Review

Review actions:

- **Approve:** Changes look good, ready to merge
- **Request changes:** Issues found, needs fixes
- **Comment:** General feedback or questions



GitHub Review

Best practices:

- Be constructive and specific
- Explain the "why" behind suggestions
- Acknowledge good work
- Review promptly
- Test the changes if possible

GitHub Issues

Issues are used to track bugs, feature requests, and tasks.

- Have a title, a description, and a discussion thread
- Can be assigned to people
- Can be labelled (`bug`, `enhancement`, `question`, ...)
- A pull request can **close** an issue automatically

Discussions are for community conversations and Q&A.

GitHub Projects

GitHub Projects is a project management tool integrated with your repositories.

What it provides:

- Kanban boards, tables, and roadmap views
- Automation and workflows
- Integration with issues and pull requests
- Progress tracking and reporting
- Team collaboration features

GitHub Projects

Use cases:

- Sprint planning
- Feature roadmaps
- Bug triage
- Release planning
- Team coordination

This is what you will use to run your project sprints.

GitHub Projects - Boards & Views

Board views:

- **Kanban:** Visual workflow (To Do → In Progress → Done)
- **Table:** Spreadsheet-like view with filters
- **Roadmap:** Timeline view for planning

GitHub Projects

Automation:

- Auto-move items when PR is merged
- Set status based on labels
- Assign items automatically
- Create items from templates

Customization:

- Custom fields (priority, effort, etc.)
- Multiple views for different purposes

Part 1.5

VS Code

Where do you write the code?

So far we talked about where code **lives** and how you **share** it.

You still need something to actually write it in.

Broadly, three options:

- A **text editor**
- An **IDE**
- A **notebook**

Text editors

A program that edits plain text. Nothing more.

- **nano, vim, emacs**: in the terminal
- **Notepad, gedit, Sublime Text, Notepad++**: graphical

Why you still care:

- They are on *every* machine, including a server you SSH into
- They start instantly
- Knowing minimal `vim` (`i` to type, `Esc`, `:wq` to save) will get you out of trouble one day. Git drops you into it to write commit

IDEs

Integrated Development Environment: an editor plus everything else you need, in one place.

Examples: **PyCharm, IntelliJ IDEA, Visual Studio, Eclipse**

IDEs

Typically:

- Syntax highlighting and **autocomplete** that understands your code
- **Debugger**: stop execution, inspect variables
- Run and test your code without leaving the window
- **Refactoring**: rename a function everywhere at once
- Integrated terminal and **version control**
- **LLM** integration with chatbot
- **CLI** (Command Line Interface)

Editor vs IDE vs Notebook

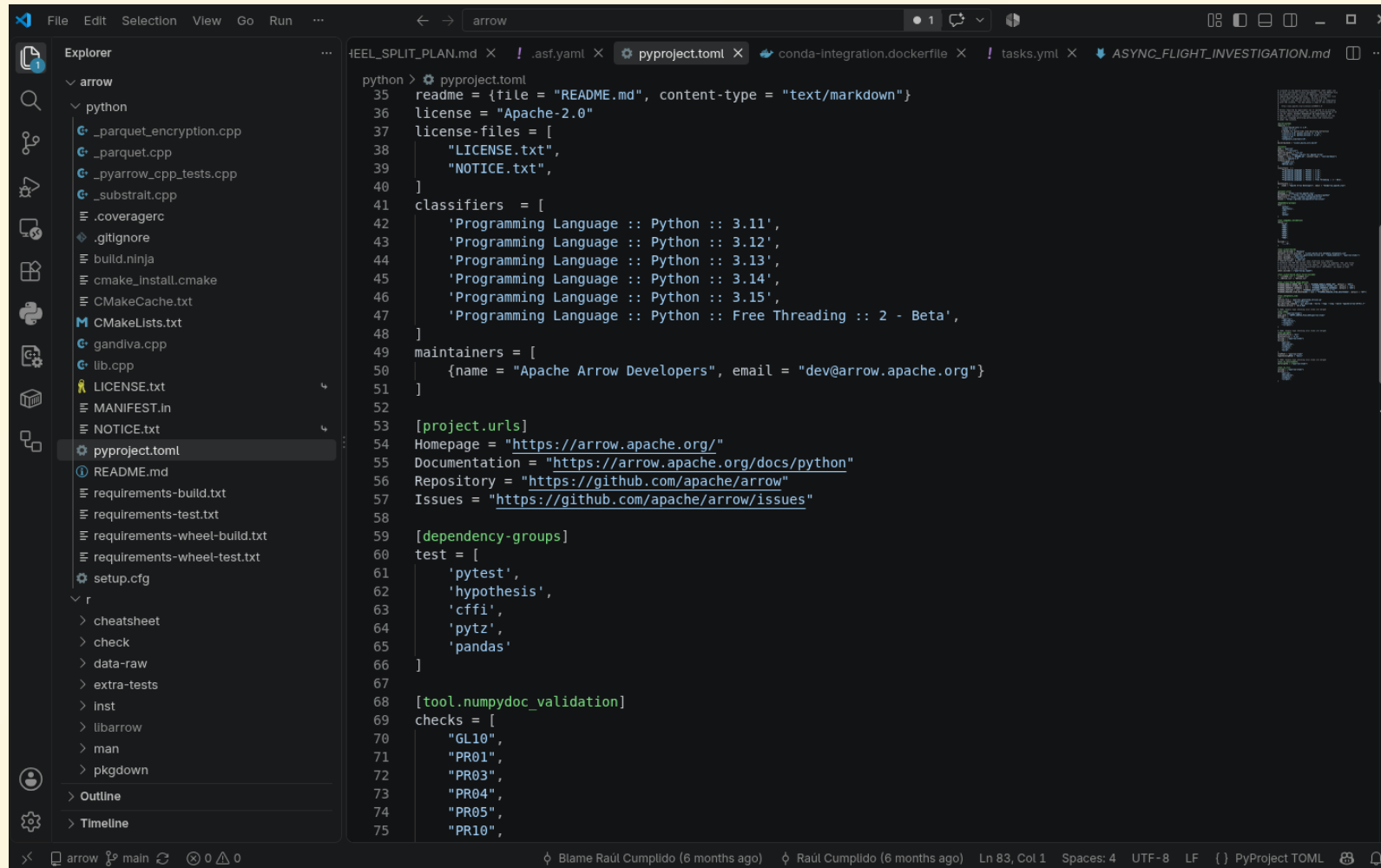
	Text editor	IDE	Notebook
Startup	Instant	Slow	Medium
Understands your code	No	Yes	Partly
Debugger	No	Yes	Limited
Good for	Quick fixes, servers	Real projects	Exploration
Version control	Awkward	Built in	Painful

Editor vs IDE vs Notebook

Notebooks are great for **exploring**. They are bad for anything that has to be reviewed, tested, or run twice the same way.

Part of this course is moving your work out of the notebook.

Visual Studio Code



Visual Studio Code

A free, open-source editor from Microsoft that sits **between** a text editor and a full IDE.

- Starts fast like an editor
- Becomes an IDE through **extensions**
- Same tool for Python, SQL, Dockerfiles, YAML, Markdown
- Runs on Linux, macOS and Windows
- By far the most used editor in the industry today

Not the only good choice but it is the one we will use, and the one you are most likely to find at work.

VS Code: the parts you will use

- **Explorer:** your project files
- **Source Control:** Git, built in; no extension needed
- **Extensions:** everything else
- **Command Palette:** `Ctrl+Shift+P`: *every* command, searchable

If you learn one shortcut today, learn the Command Palette. You do not need to remember where anything is in the menus.

VS Code and Git

The **Source Control** panel gives you:

- The list of modified files, with a side-by-side **diff** on click
- Stage a file, or stage **individual lines** of a file
- Write a commit message and commit
- Current branch and sync status in the bottom-left corner
- **Conflict resolution** with "Accept Current / Incoming / Both"

We will use it in a moment.

Part 2

Hands-on

What we are going to do

1. Create a GitHub account
2. Install Git
3. Configure Git
4. Set up authentication (SSH key)
5. Install and set up Visual Studio Code

What we are going to do

6. Create a repository on GitHub
7. Clone it
8. Create a branch, make a change, commit
9. Push and open a pull request
10. Review someone else's pull request
11. Issues and Projects

Shout if you get stuck: everyone needs to finish 1 to 5.

1. GitHub account

Go to <https://github.com> and sign up.

- Pick a **username you will not be embarrassed by**: this is a public portfolio, recruiters look at it
- Use an email you will keep after university
- Enable **two-factor authentication** (GitHub requires it)

Students: apply for the **GitHub Student Developer Pack** (<https://education.github.com/pack>): free private repos, Copilot, and credits for other services.

2. Install Git

<https://docs.github.com/en/get-started/git-basics/set-up-git>

Linux (Debian/Ubuntu)

```
sudo apt install git
```

macOS

```
brew install git
```

(or just run `git` and accept the Xcode command line tools prompt)

2. Install Git

Windows

Download from <https://git-scm.com/download/win>, which also gives you

Git Bash.

Check it worked:

```
git --version
```

3. Configure Git

Git puts your name and email in **every commit you make**. Set them once:

```
git config --global user.name "Your Name"  
git config --global user.email "your@email.com"
```

Use the **same email as your GitHub account**, otherwise your commits will not be linked to your profile.

3. Configure Git

While we are here:

```
git config --global init.defaultBranch main  
git config --global pull.rebase false
```

Check what you have:

```
git config --global --list
```

4. Authentication: SSH keys

<https://docs.github.com/en/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent>

GitHub might not accept passwords from the command line. Use an SSH key.

Look on Google for "How to set up my SSH key on GitHub" and look at your OS.

4. Authentication: SSH keys

Linux/macOS Generate one:

```
ssh-keygen -t ed25519 -C "your@email.com"
```

Press Enter for the default location. A passphrase is optional but recommended.

Copy the public key:

```
cat ~/.ssh/id_ed25519.pub
```

4. Authentication: SSH keys

Add it to GitHub:

Settings → SSH and GPG keys → New SSH key → paste → Add

Test it:

```
ssh -T git@github.com
```

You should see:

```
Hi <username>! You've successfully authenticated, but GitHub does not  
provide shell access.
```

5. Install VS Code

Linux (Debian/Ubuntu)

```
sudo snap install code --classic
```

or download the `.deb` from <https://code.visualstudio.com/>

macOS

```
brew install --cask visual-studio-code
```

5. Install VS Code

Windows

Download the installer from <https://code.visualstudio.com/>.

Tick **"Add to PATH"** during installation.

5. Check it works

Open a terminal and run:

```
code --version
```

If `code` is not found on macOS: open VS Code, `Cmd+Shift+P` → *"Shell Command: Install 'code' command in PATH"*.

Now open a folder with it:

```
code .
```

5. Set up VS Code

Install the Python extension:

Extensions panel (`Ctrl+Shift+X`) → search `Python` → Install the one from **Microsoft**.

Do the same for **Jupyter** if you want notebooks inside the editor.

Try the Command Palette: `Ctrl+Shift+P`, type `theme`, change the colour theme. This is how you will do everything else.

5. VS Code and Git

Git support is **built in**, no extension needed. Open the cloned repo (next step) and check the **Source Control** panel (branch icon):

- Modified files, with a diff when you click one
- Stage, write a message, commit
- Branch name and sync status, bottom-left corner

Optional but useful: the **GitHub Pull Requests** extension lets you review PRs without leaving the editor.

5. A warning

The GUI is comfortable, and you should use it. But **learn the commands**

first. When something goes wrong (and it will) the error message you find online will be about `git rebase --abort`, not about a button.

6. Create a repository on GitHub

Top-right + → **New repository**, or go to <https://github.com/new>.

- **Owner:** your user
- **Repository name:** `ads-<your-username>` (no spaces)
- **Description:** one line, optional
- **Public** or **Private:** public is fine for this

6. Create a repository on GitHub

Before you click *Create*, tick **Add a README file**.

An empty repository has no commits, which makes the next steps more annoying than they need to be. The README gives you a first commit to clone.

6. Create a repository on GitHub

Also worth adding:

- **.gitignore** → choose the **Python** template. Stops you committing **__pycache__**, **.venv**, checkpoints...
- **License** → **MIT** if you want others to be able to use your code

Click **Create repository**.

6. What you just made

An empty-ish repository, living **only on GitHub**.

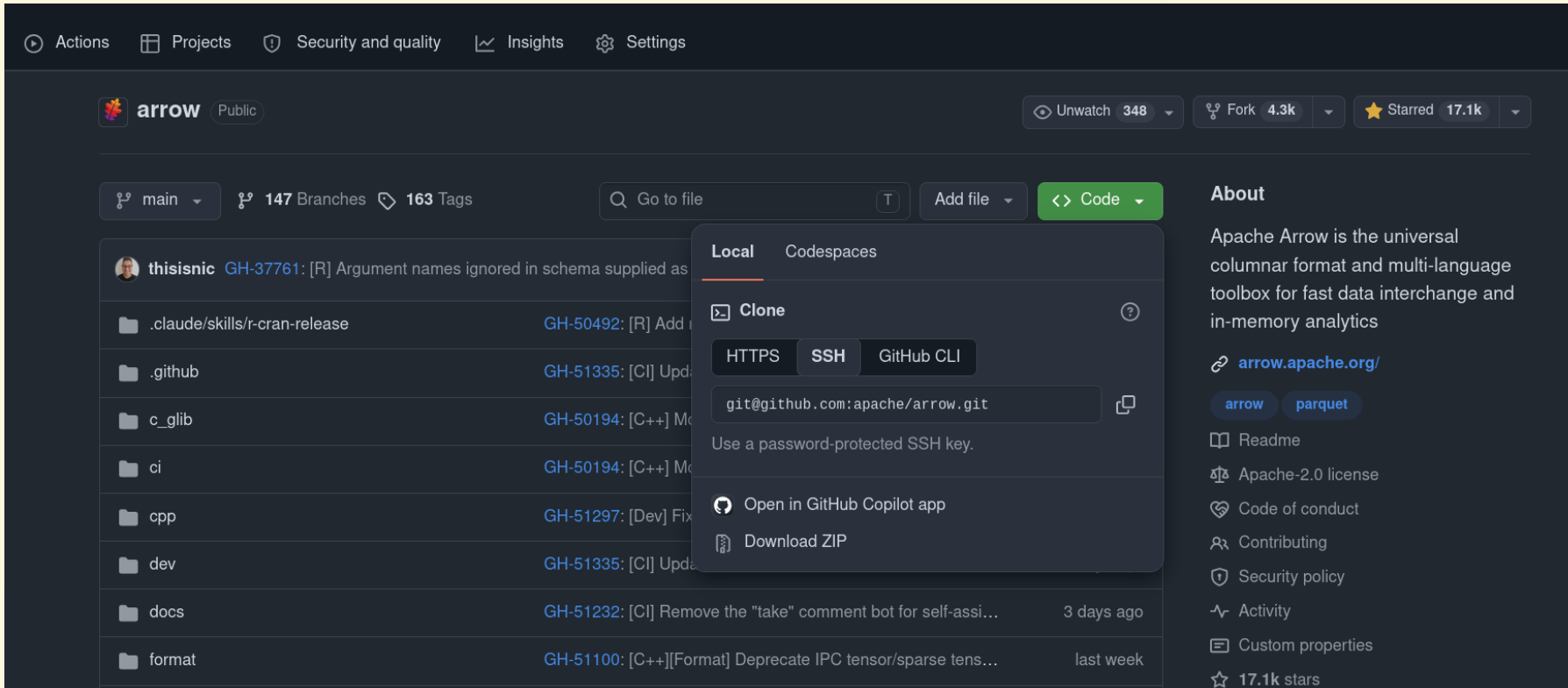
```
GitHub:  main with README.md and (potentially) License files  
Laptop:  (nothing yet)
```

Next step is to get a copy onto your machine.

Note: a repository can also start on your laptop with `git init`, and be connected to GitHub afterwards with `git remote add origin ...`. Starting on GitHub is fewer steps, so that is what we do today.

7. Clone a repository

Get the URL from the green **Code** button on GitHub, SSH tab.



The screenshot shows the Apache Arrow repository on GitHub. The 'Code' button is clicked, opening a dropdown menu. The 'SSH' tab is selected, displaying the URL `git@github.com:apache/arrow.git`. The repository page includes a file list on the left, repository statistics at the top, and an 'About' section on the right.

Repository Statistics:

- Unwatch: 348
- Fork: 4.3k
- Starred: 17.1k

File List:

File Name	Commit Hash	Commit Message	Time
.claude/skills/r-cran-release	GH-50492	[R] Add	
.github	GH-51335	[CI] Upd	
c_glib	GH-50194	[C++] M	
ci	GH-50194	[C++] M	
cpp	GH-51297	[Dev] Fix	
dev	GH-51335	[CI] Upd	
docs	GH-51232	[CI] Remove the "take" comment bot for self-assi...	3 days ago
format	GH-51100	[C++][Format] Deprecate IPC tensor/sparse tens...	last week

About:

- Apache Arrow is the universal columnar format and multi-language toolbox for fast data interchange and in-memory analytics.
- arrow.apache.org/
- arrow
- parquet
- Readme
- Apache-2.0 license
- Code of conduct
- Contributing
- Security policy
- Activity
- Custom properties
- 17.1k stars

7. Clone a repository

```
git clone git@github.com:<owner>/<repo>.git  
cd <repo>
```

Look at what you got:

```
ls -a          # there is a .git folder  
git status     # clean working tree  
git log --oneline -5  
git remote -v  # origin points to GitHub
```

Question: what is the difference between `git init` and `git clone`?

8. Branch, change, commit

```
git switch -c add-<your-username>
```

Make your change (add a file with your name, edit the README, ...).

```
git status          # see what changed
git diff            # see the actual change
git add <file>
git status          # now it is staged
git commit -m "Add <your name> to the participants list"
git log --oneline -3
```

9. Push and open a pull request

```
git push -u origin add-<your-username>
```

Git prints a link to open the PR. Or go to GitHub (it shows a banner).

On the PR:

- Give it a clear title
- Describe *what* you changed and *why*
- Look at the **Files changed** tab before asking anyone to review it

10. Review a pull request

Find a classmate's PR.

- Go to **Files changed**
- Click a line number to leave a comment
- **Review changes** → Comment / Approve / Request changes

Leave at least one **useful** comment. "lgtm" is not a review.

11. Issues and Projects

- Open an **Issue** describing something to improve
- Create a **Project** board for your repository
- Add the issue to the board, move it across columns
- Link a pull request to the issue (`Closes #12` in the PR description)

This is exactly the workflow you will use for your sprints.

If you get stuck

```
git status
```

Read what it says. Git is unusually good at telling you what to do next:

- "use git restore --staged <file>..."
- "use git add <file>... to mark resolution"
- "use git push to publish your local commits"

Second thing to try:

```
git log --oneline --graph --all
```

If you want to learn more about Git...

Extra material

Extra: this part is not required

Everything from here on is **not** needed to pass the course or to do the project.

It is here because at some point you will hit one of these, and you will want to know what is going on.

Fast-forward merge

Occurs when the target branch hasn't moved since the feature branch was created.

Example:

```
main:      A---B
           |
feature:    C---D
After merge: A---B---C---D (main)
```

Characteristics:

- No new merge commit is created

3-way merge

Occurs when both branches have diverged with new commits.

Example:

```
main:      A---B---E
            \
feature:    C---D
After merge: A---B---E---M (main)
            \      /
              C---D
```

Characteristics:

- Creates a new merge commit

Git rebase

Rebasing replays commits from one branch onto another branch, creating a linear history.

Example:

```
Before:      A---B---E (main)
              \
feature:      C---D

After:       A---B---E---C'---D' (feature)
```

Git rebase

Basic workflow:

1. Switch to feature branch: `git checkout feature-branch`
2. Rebase onto main: `git rebase main`

What happens:

- Commits C and D are "replayed" on top of E
- Creates new commits C' and D' (different SHA hashes)
- Original C and D commits are discarded

Merge vs Rebase

Feature	Merge	Rebase
History	Preserves branching history	Creates linear history
Commits	Creates merge commits	Rewrites commit history
Safety	Non-destructive	Can be destructive
Use case	Shared/public branches	Private/local branches
Conflicts	Resolve once on merge	May resolve more times
Command	<code>git merge feature</code>	<code>git rebase main</code>

Git HEAD

HEAD is a pointer that refers to the current branch and commit you're working on.

Key concepts:

- Points to the current branch (usually)
- Represents your current working position
- Moves when you make commits or switch branches
- Can be "detached" when pointing directly to a commit

Git HEAD

Common scenarios:

- `HEAD -> main` - Currently on main branch
- `HEAD -> feature` - Currently on feature branch
- `HEAD at abc1234` - Detached HEAD at specific commit

Related commands:

- `git log --oneline` - Shows HEAD position
- `git checkout HEAD~1` - Move HEAD back one commit
- `git reset HEAD` - Reset staging area to HEAD

Git tag

Tags are labels that mark specific points in your repository's history, typically used for releases.

Types of tags:

- **Lightweight tag:** Just a pointer to a commit
- **Annotated tag:** Contains metadata (author, date, message)

Git tag

Common commands:

- `git tag v1.0` - Create lightweight tag
- `git tag -a v1.0 -m "Release 1.0"` - Create annotated tag
- `git tag` - List all tags
- `git show v1.0` - Show tag information
- `git push origin v1.0` - Push tag to remote

Use cases:

- Mark software releases (v1.0, v2.1, etc.)

Conflict markers in detail

- <<<<<< HEAD - Start of current branch changes
- ===== - Separator between changes
- >>>>>> feature-branch - End of other branch changes

Resolved version:

```
def calculate_total(items):  
    # Calculate total with tax and discount  
    subtotal = sum(item.price for item in items)  
    discount = subtotal * 0.1 # 10% discount  
    total_with_discount = subtotal - discount  
    return total_with_discount * 1.08 # 8% tax
```

GitHub Conflicts

GitHub conflicts occur when a pull request can't be automatically merged.

When conflicts happen:

- Someone else modified the same lines you changed
- Changes were made to main branch after you created your feature branch
- Multiple people working on similar files

Resolution options:

GitHub Merge options

Merging on GitHub incorporates pull request changes into the target branch.

- **Create a merge commit:** Preserves branch history
- **Squash and merge:** Combines all commits into one
- **Rebase and merge:** Replays commits without merge commit

Best practices:

- Delete merged branches to keep repo clean
- Use descriptive merge commit messages

Where to learn more

- **Pro Git** book, free online: <https://git-scm.com/book> — the reference
- <https://learngitbranching.js.org/> — interactive, visual, genuinely good
- <https://ohshitgit.com/> — how to undo the thing you just did
- `git help <command>` — the manual is offline and surprisingly readable

Questions?

See you on 28/09 with Docker